ListNode curr = head;

ListNode prev = null;
while (curr!= null) {
ListNode ahead = curr.next;
curr.next = prev;
prev = curr;
curr = ahead;
}

return prev;

public ListNode sortList(ListNode head) { if (head == null || head.next == null) { return head;
}

ListNode mid = middleNode(head);
ListNode midnext = mid.next;
mid.next = null;
ListNode A = sortList(head);
ListNode B = sortList(midnext);
return mergeTwoLists(A, B);

}

public ListNode middleNode(ListNode head) { ListNode slow = head;
ListNode fast = head;
while (fast!= null && fast.next!= null) { slow = slow.next;
fast = fast.next.next;
}
return slow;
}

while(head!=null) {
// k element stack push krna hai
int c=0;
while(c<k) {
st.push(head);
head=head.next;
}

// k element ko stack pop krke attach krna hai

}

public ListNode reverseKGroup(ListNode head, int k) { Stack<ListNode> st = new Stack<>();
ListNode dummy = new ListNode();
ListNode temp = dummy;
while (head!= null) {
// k element stack push krna hai
while (head!= null && st.size() < k) {
st.push(head);
head = head.next;
}

// k element ko stack pop krke attach krna hai if (st.size() == k) {
while (!st.isEmpty()) {
dummy.next = st.pop();
dummy = dummy.next;
}
}

}
}

public ListNode reverseKGroup(ListNode head, int k) { Stack<ListNode> st = new Stack<>();
ListNode dummy = new ListNode();
ListNode temp = dummy;
ListNode prevstart=head;
while (head!= null) {
// k element stack push krna hai
while (head!= null && st.size() < k) {
st.push(head);
head = head.next;
}

// k element ko stack pop krke attach krna hai if (st.size() == k) {
prevstart=head;
while (!st.isEmpty()) {
dummy.next = st.pop();
dummy = dummy.next;
}
}

}
}

Stack<ListNode> st = new Stack<>();
ListNode dummy = new ListNode();
ListNode temp = dummy;
ListNode prevstart=head;
while (head!= null) {
// k element stack push krna hai
while (head!= null && st.size() < k) {
st.push(head);
head = head.next;
}

// k element ko stack pop krke attach krna hai if (st.size() == k) {
prevstart=head;
while (!st.isEmpty()) {
dummy.next = st.pop();
dummy = dummy.next;
}
}
else {
dummy.next=prevstart;
}

}
return temp.next;

public ListNode reverseKGroup(ListNode head, int k) { Stack<ListNode> st = new Stack<>();
ListNode dummy = new ListNode();
ListNode temp = dummy;
ListNode prevstart=head;
while (head!= null) {
// k element stack push krna hai
while (head!= null && st.size() < k) {
st.push(head);
head = head.next;
}

// k element ko stack pop krke attach krna hai if (st.size() == k) {
prevstart.next=null;
prevstart=head;
while (!st.isEmpty()) {
dummy.next = st.pop();
dummy = dummy.next;
}
}
else {
dummy.next=prevstart;
}

}
return temp.next;
}

Lec-27

12/04/26 4:45 PM